

Big Data Collection: Big data collection is the process of gathering large amounts of data from various sources such as sensors, websites, devices, social media, logs and applications so it can be stored and analyzed for insights.

Strategies:

1.Automated Data Collection: This strategy uses software, sensors, or tools to collect data automatically without human effort. It ensures fast, continuous, and accurate data gathering from sources like websites, IoT devices, logs, and applications.

2.Batch Processing: Batch processing collects data over a period of time and processes it all at once. It is useful when real-time results are not required, such as generating daily reports or processing large historical datasets.

3.Real-Time Data Processing: Real-time processing handles data the moment it arrives. It is used when immediate actions or insights are needed, such as fraud detection, live traffic updates, or sensor alerts.

4.Data Integration: Data integration combines data from multiple sources into a single, unified view. It helps make the data consistent and easier to analyzed by removing differences between formats, systems, or structures.

5.Data Cleaning: Data cleaning improves data quality by correcting errors, removing duplicates, handling missing values, and fixing inconsistencies. Clean data is essential for accurate analysis and decision-making.

6.Cloud Storage: Cloud storage stores data on remote servers managed by cloud providers. It allows unlimited scalability, easy access, lower costs, and reliable backup, making it ideal for big data storage.

Types of Data Sources:

1.Social Media: Data generated from platforms like Facebook, Twitter, Instagram and YouTube.

Example: Likes, shares and comments on a post.

How it is used: To analyze user behavior, trends, customer sentiments and marketing performance.

2.Sensor Data: Data collected from physical sensors in devices, machines, vehicles or IoT systems.

Example: A temperature sensor recording heat levels.

How it is used: For monitoring environments, tracking machine health, automation and real-time alerts.

3.Transaction Data: Data produced during business transactions such as purchases, payments, orders or banking activities.

Example: A supermarket billing system storing purchased items.

How it is used: For sales analysis, forecasting, fraud detection and financial reporting.

4.Web Data: Data collected from websites through logs, cookies, clickstreams or web scraping.

Example: A website tracking which pages a user clicks.

How it is used: To understand website traffic, improve user experience and personalize content.

5.App Data: Data generated by mobile or desktop applications, including user behavior, app usage patterns and in-app actions.

Example: A food delivery app storing order history.

How it is used: To improve features, personalize recommendations and track app performance.

6.Enterprise Data: Internal data stored within an organization, such as employee records, CRM data, sales reports, inventory data and financial data.

Example: Employee attendance records in a company.

How it is used: For business operations, decision-making, HR management and performance analysis.

7.Public Data: Open data available to everyone from governments, research institutions or public databases.

Example: Government census data.

How it is used: For research, planning, policy-making and demographic analysis.

8.Multimedia Data: Unstructured data in the form of images, videos, audio files, animations and graphics.

Example: CCTV video recordings.

How it is used: For surveillance, facial recognition, medical imaging and media analytics.

Extract, Transformation, Load (ETL):

1.Extraction: The Extraction phase is the first step in the ETL process, where raw data is collected from different types of data sources. These sources may be structured (such as SQL databases, ERPs and CRMs), semi-structured (such as JSON or XML) or unstructured (such as emails, flat files or web pages).

The main purpose of extraction is to gather data **without modifying its original format** so that it can be processed in the next stages of ETL.

2.Transformation: The Transformation phase is where raw and inconsistent data is converted into a clean and usable form. This step ensures the data follows business rules and meets quality standards required for accurate analysis.

Common transformation operations include:

- **Data Filtering:** Removing incorrect or unnecessary data
- **Data Sorting:** Arranging data in a required order
- **Data Aggregation:** Summarizing data for meaningful insights

More advanced transformations may involve tasks like currency conversion, text normalization or applying domain-specific rules to prepare the data according to organizational needs.

3.Loading: The Loading phase transfers the cleaned and transformed data into a target system such as a data warehouse or data lake.

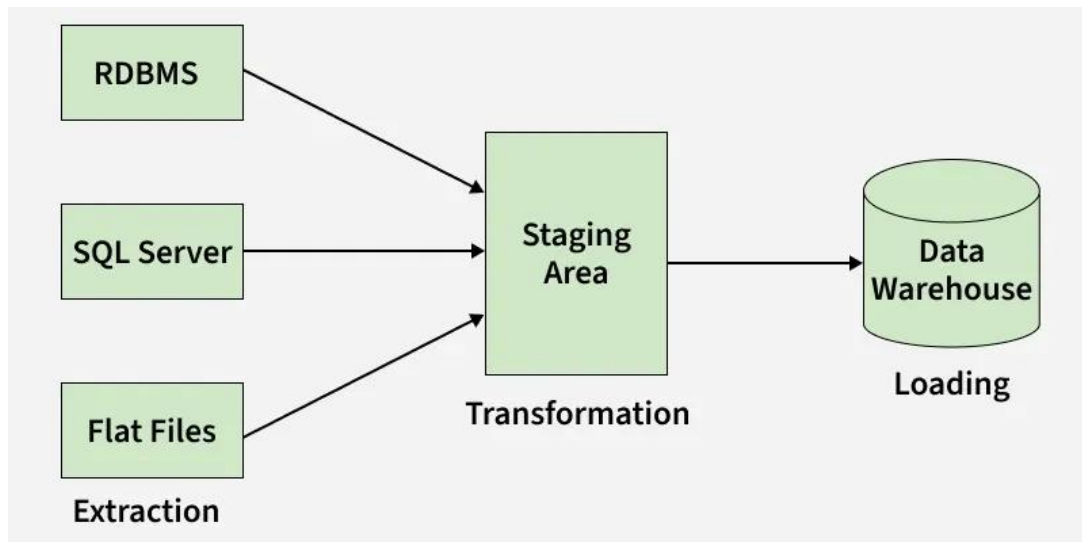
There are two main loading methods:

- **Full Load:** Loads all available data into the system, usually during the first load
- **Incremental Load:** Loads only new or updated data, making it more efficient for regular updates

Pipelining in ETL: Pipelining in ETL means that extraction, transformation and loading occur in **overlapping stages** instead of one after the other. As soon as some data is extracted, it can start transforming, and transformed data can be loaded while new data is still being processed.

This parallel execution:

- Speeds up the entire ETL process
- Reduces waiting time
- Improves system resource usage
- Makes the ETL workflow more scalable

**Extract, Load, Transform (ELT):**

1.Extraction: In the Extraction phase of ELT, raw data is collected from a wide variety of data sources. These sources can include structured systems such as SQL databases, ERP or CRM systems, semi-structured formats like JSON or XML and unstructured sources such as emails, logs or multimedia files. The goal of extraction in ELT is to gather all required data in its original form without performing any processing at this stage.

2.Loading: In ELT, the next step after extraction is to load the raw data directly into a target system, usually a modern data warehouse or data lake. These storage systems are designed to handle large volumes of raw data efficiently. Because data is loaded before transformation, organizations can store huge amounts of information quickly, allowing analysts and systems to access the raw data whenever needed.

Two common loading approaches include:

- Full Load: Loading all extracted data into the target system
- Incremental Load: Loading only newly added or updated data

3.Transformation: The Transformation phase in ELT is performed inside the target system after the data has been loaded. Modern data warehouses (like Snowflake, BigQuery or Redshift) provide powerful computing capabilities, allowing complex transformations to be done directly within them. Common transformations include:

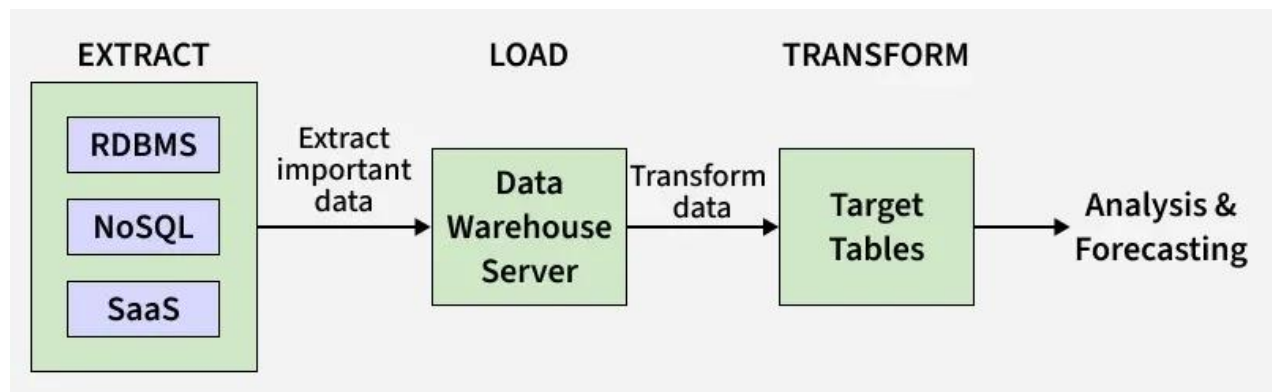
- Data cleaning
- Data filtering and sorting
- Aggregations such as totals, averages or summaries
- Conversions, formatting changes or applying business rules

Performing transformations inside the warehouse allows faster processing, better scalability and flexibility to reprocess data whenever needed.

Pipelining in ELT: In ELT, data flows in a way that prioritizes fast loading followed by on-demand transformation. Because raw data is stored as-is, different teams can run their own transformations without affecting the loading process.

This design:

- Speeds up the initial loading
- Uses the power of modern cloud warehouses for processing
- Offers more flexibility for analytics and machine learning

**ETL vs ELT:**

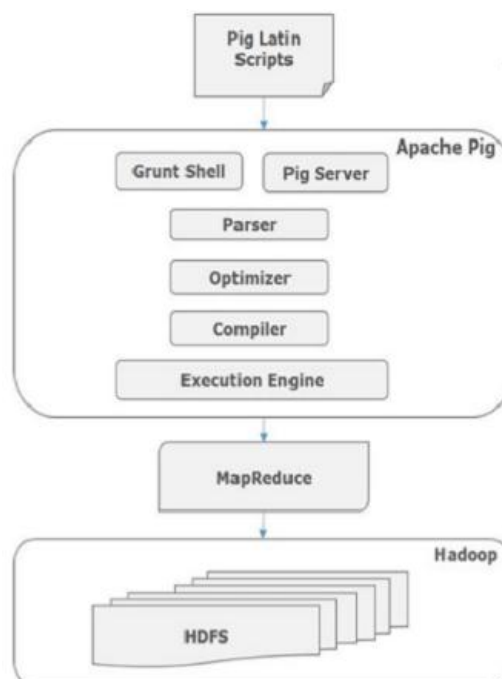
Category	ETL	ELT
Acronym Meaning	Extract, Transform, Load	Extract, Load, Transform
Definition	Extracts raw data, transforms it on a secondary server, then loads it into the destination.	Extracts raw data, loads it directly into the destination and transforms it there.
Processing Speed	Slower; data transformation occurs before loading.	Faster; data is loaded first and transformed in parallel.
Data Volume	Best for smaller, complex data sets like marketing data.	Suited for large data sets requiring speed, like real-time analytics.
Data Output	Primarily structured data.	Structured, semi-structured and unstructured data.
Data Lake Compatibility	Not compatible with data lakes.	Fully compatible with data lakes.
Cost Efficiency	Higher costs due to the need for separate servers and processing infrastructure.	More cost-effective, leveraging cloud resources for scalability.
Flexibility	Best for structured data transformation.	Handles structured and unstructured data with ease.

Pig Introduction:

1. Pig uses a scripting language called Pig Latin, which is easier to write and understand compared to Java MapReduce code.
2. Before Pig dev's have to write long complex programs in Java.
3. Pig solves this by providing a simple SQL like scripting language.
4. Pig Latin is a data flow language where you write steps describing how data should move and transform like float data, filter data, group it, join it, sort it, save it and dump it.

Features of Apache Pig:

1. **High-Level Language:** Pig uses a scripting language called Pig Latin, which provides a high-level abstraction for data processing. It allows users to focus on the overall logic instead of writing complex MapReduce programs.
2. **Requires Less Code:** Pig scripts are concise and require far fewer lines of code compared to traditional MapReduce. This reduces development time and makes data processing tasks easier to write, read and maintain.
3. **Handles All Types of Data:** Apache Pig can process structured, semi-structured and unstructured data. Whether dealing with logs, text files, images or tabular datasets, Pig offers flexible data handling capabilities.
4. **Extensible:** Pig supports user-defined functions (UDFs) that allow developers to add custom logic in languages like Java, Python or JavaScript. This makes Pig easily extensible for specialized processing needs.
5. **Automatic Optimization:** Pig includes an internal optimization engine that automatically improves the execution plan. This means Pig figures out the best way to run a script without requiring users to manually tune performance.
6. **Runs Everywhere:** Apache Pig can run in multiple execution environments including local mode, MapReduce mode and Tez mode. This flexibility allows Pig to work on a single machine or a large Hadoop cluster.

Pig Architecture:

1. The language used to analyze data in Hadoop using Pig is known as Pig Latin.
2. It is a high-level data processing language which provides a rich set of data types and operators to perform various operations on the data.
3. To perform a particular task, programmers using Pig, need to write a Pig script using the Pig Latin language, and execute them using any of the execution mechanisms (Grunt Shell, UDFs, Embedded).
4. After execution, these scripts will go through a series of transformations applied by the Pig Framework, to produce the desired output.
5. Internally, Apache Pig converts these scripts into a series of MapReduce jobs, and thus, it makes the programmers' job easy.

Apache Pig Components: As shown in the figure, there are various components in the Apache Pig framework.

Parser: Initially the Pig Scripts are handled by the Parser. It checks the syntax of the script, does type checking, and other miscellaneous checks. The output of the parser will be a DAG (directed acyclic graph), which represents the Pig Latin statements and logical operators. In the DAG, the logical operators of the script are represented as the nodes and the data flows are represented as edges.

Optimizer: The logical plan (DAG) is passed to the logical optimizer, which carries out the logical optimizations such as projection and pushdown.

Compiler: The compiler compiles the optimized logical plan into a series of MapReduce jobs.

Execution engine: Finally, the MapReduce jobs are submitted to Hadoop in a sorted order. Finally, these MapReduce jobs are executed on Hadoop producing the desired results.

Pig Infrastructure Requirement for Big Data:

1. **Massive Storage Capacity:** Big data systems must store huge amounts of structured, semi-structured and unstructured data. The storage infrastructure should support petabytes of data without performance degradation.
2. **High-Speed Read and Write:** Since big data applications often involve continuous data flow, the system must allow fast data ingestion and quick retrieval. High read and write speeds ensure smooth processing and timely analysis.
3. **Distributed Storage:** Big data is usually stored across multiple machines rather than a single server. Distributed storage improves performance by allowing parallel access and ensures reliability through data replication.
4. **Fault Tolerance:** The storage system must continue working even if some nodes fail. Techniques like data replication and automatic recovery help prevent data loss and maintain system availability.
5. **Scalability:** As data grows, the storage system should be able to expand easily. Horizontal scalability allows new nodes to be added without disrupting operations.
6. **Cost-Effectiveness:** Big data storage should be affordable to maintain. Using commodity hardware, cloud storage and efficient data management techniques helps reduce overall costs.
7. **Security:** Since big data may contain sensitive information, the infrastructure must support strong security measures such as encryption, authentication and access control to protect data from unauthorized access.
8. **Data Integration:** The storage system should support integration of data coming from various sources and formats. This helps in creating a unified and consistent dataset for analysis.

Pig Execution Mode: Pig supports two main execution modes that determine how Pig scripts are processed: Local Mode and MapReduce Mode. These modes allow users to choose between running Pig on a single machine or on a distributed Hadoop cluster, depending on the size of the data and the purpose of execution.

1.Local Mode: In Local Mode, Pig runs entirely on a single local machine. Both the Pig engine and the data processing take place within the local filesystem. This mode does not require Hadoop or HDFS, making it useful for testing, debugging, and working with small datasets. Since all operations execute locally, the setup is simple and quick.

Command: pig -x local

2.MapReduce Mode: MapReduce Mode is Pig's default execution mode. In this mode, Pig scripts are automatically converted into a series of MapReduce jobs and executed on a Hadoop cluster. This allows Pig to process large-scale datasets in a distributed manner using HDFS for storage. It is suitable for production environments where high performance, fault tolerance and scalability are required.

Command: pig -x mapreduce

Collection Model: A collection method is the way data is gathered from a system or network by observing, recording and capturing activities such as logs, sensor data or network packets so the data can be stored and analyzed later.

Collection methods help in:

- Watching network traffic as it moves
- Recording what happens on a computer or network
- Saving logs and packets for future analysis

Common Collection Methods Include:

- Reading log files
- Using sensors
- Capturing network packets

Log Files:

- 1.Log files are system-generated files that maintain a chronological record of events occurring in a computer system or network.
- 2.They are automatically created by operating systems, applications and network devices such as servers, routers and firewalls.
- 3.Log files record information related to system activities, user actions, errors and security events, usually with time and date details.
- 4.They act as a source of historical data for monitoring, analysis and auditing.
- 5.Log files are widely used for system monitoring, troubleshooting and performance analysis.
- 6.In security, they help in detecting unauthorized access and abnormal activities.
- 7.**Limitation:** Log files capture only those events that the system is configured to record.

Sensors:

- 1.Sensors are hardware or software components used to monitor and detect events or changes in a system, environment or network.
- 2.They continuously collect data such as activity levels, traffic patterns or physical conditions.
- 3.Sensors automatically send the collected data to a system for analysis and alert generation.
- 4.In computing and networking, sensors are often used for monitoring, security and real-time detection.

5.They improve system reliability and security by continuously observing activities and reporting unusual behavior.

6.Ex: An intrusion detection sensor monitors network traffic and raises an alert when it detects multiple failed login attempts, indicating a possible attack.

Capturing network packets: This is the process of intercepting and recording data packets as they travel across a computer network. It allows administrators or analysts to examine the contents of packets to monitor network activity, troubleshoot problems, analyze performance and detect security threats.

Methods for acquiring network data: Methods for acquiring network data refer to the techniques used to collect information about network activity and traffic so it can be analyzed for monitoring, performance and security purposes.

1.Libpcap-Based Packet Capture:

i. Libpcap-based packet capture uses a software library called libpcap to collect network packets.

ii. The packets are copied from the network card to the operating system and then to the capturing application.

iii. Because of these multiple copies, it works well for normal or low-speed networks.

iv. Ex: Tools like **Wireshark** use libpcap to capture packets on a home or college network to see which websites are being accessed.

2.Zero-Copy Packet Capture:

i. Zero-copy packet capture collects packets without copying them multiple times.

ii. Packets go directly from the network card to the application memory, making this method much faster and more efficient.

iii. It is used in high-speed networks where performance is very important.

iv. Ex: Internet service providers use zero-copy packet capture to monitor traffic on high-speed backbone networks without slowing them down.

Specialized network monitoring software's: Specialized network monitoring software are tools designed to observe, capture and analyze network traffic in order to ensure proper network performance, security and reliability.

Wireshark:

i. It is a network packet analyzer used to capture and analyze network traffic.

ii. It shows detailed packet information such as protocol, source and destination.

iii. It is widely used for troubleshooting and security analysis.

iv. Ex: Analyzing network traffic to find why a website is loading slowly.

SmartSniff:

i. It is a network monitoring tool that captures TCP/IP data packets.

ii. It allows viewing communication between computers in a readable format.

iii. It is useful for monitoring suspicious network activity.

iv. Ex: Monitoring data sent between two systems on a local network.

WinnetCap:

- i. It is a packet capture driver used in Windows systems.
- ii. It provides low-level access to network traffic for applications.
- iii. It supports packet capturing tools like Wireshark.
- iv. **Ex:** Enabling Wireshark to capture packets on a Windows machine.

Pig Latin Command: Pig Latin is a high-level scripting language used in Apache Pig to analyze and process large datasets stored in Hadoop. It allows users to write simple data processing scripts, which Pig automatically converts into MapReduce jobs and executes on a Hadoop cluster.

Commands:

1.LOAD & STORE: LOAD is used to bring data from a file or folder into Pig so it can be processed. It creates a relation from the loaded data. STORE is used to write the processed data into a file or directory for future use.

2.DESCRIBE: The DESCRIBE command displays the schema of a relation, showing field names and data types. It helps users understand the structure of the data. Diagnostic command.

3.DUMP: The DUMP command displays the actual contents of a relation on the screen. It is mainly used for testing and debugging purposes. Diagnostic command.

4.ILLUSTRATE: The ILLUSTRATE command shows a sample execution of a Pig script. It helps in understanding how data flows through different operations without running the entire dataset.

SQL VS Pig Latin:

CRITERIA	PIG	SQL
Language used	Pig Latin	SQL
Definition	High-level data flow language for processing big data	Declarative query language for databases
Developed by	Yahoo	IBM (later standardized by ANSI / ISO)
Suitable for	Large-scale data processing and ETL	Querying and managing relational data
Operational for	Complex data analysis and transformation	Business queries and reporting
Compatibility with MapReduce	Yes, Pig scripts are converted into MapReduce jobs	No direct MapReduce support
Schema Support	Optional (schema-on-read)	Mandatory (schema-on-write)
Mainly used by	Programmers and researchers	Data analysts, data scientists and programmers

Pig Operations: Pig operations are the different actions performed in Apache Pig to load, process, analyze and store large datasets using Pig Latin commands.

Operations:

1.LOAD: LOAD is used to read data from a file or directory into Pig so it can be processed.

Ex: A = LOAD 'student.csv' USING PigStorage(';');

2.FILTER: FILTER is used to select only those records that satisfy a given condition.

Ex: B = FILTER A BY marks > 60;

3.ORDER: ORDER is used to sort the data in ascending or descending order.

Ex: C = ORDER A BY marks DESC;

4.GROUP: GROUP is used to group data based on a common field for aggregation.

Ex: D = GROUP A BY class;

5.CO-GROUP: CO-GROUP is used to group data from two or more relations based on a common key.

Ex: E = COGROUP A BY id, B BY id;

What is batch mode in Pig?

1. When you write a Pig script with multiple commands and then run all of them together as a batch.

2. Pig executes all the commands in the script one by one automatically. You don't need to write/type the command one at a time.

3. It is useful when you have many steps to process large datasets.

Pig Execution mode:

1.Batch Mode: Batch mode in Pig is a mode of execution where Pig Latin commands are written in a script file and the entire script is executed together in one run, instead of entering commands one by one. It is mainly used for processing large datasets and running automated jobs.

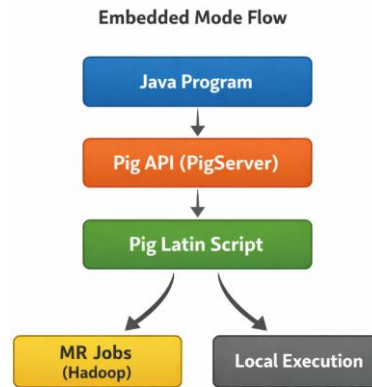
i. Local Mode: In Local Mode, Pig runs on a single local machine and processes data from the local file system. It does not require Hadoop or HDFS and is mainly used for testing, debugging and small datasets. Command: **pig -x local**

ii. MapReduce Mode: MapReduce Mode is Pig's default mode where Pig scripts are converted into MapReduce jobs and executed on a Hadoop cluster using HDFS. It is suitable for large datasets and production environments. Command: **pig -x mapreduce**

Why it is called batch mode: It is called batch mode because Pig Latin commands are written in a script file and executed together as a single batch, rather than being run one by one interactively. The entire set of operations is submitted at once, making it suitable for large data processing and automated jobs.

2.Embedded Mode: Embedded mode in Pig means running Pig inside another program, like a Java application, instead of running it from the Pig shell. It is used when Pig processing needs to be part of an application.

How embedded mode works: In embedded mode, Pig runs inside another application, usually a Java program. The application sends Pig Latin commands to Pig programmatically instead of using the Pig shell by using Java APIs. Execution still happens in Local or MapReduce mode. Pig processes the data in the background and returns the results to the application. It depends on how pig is invoked.



1. Java Program: This is the controller of the entire operation. A normal Java application is written, and it controls how and what data is processed. In embedded mode, Pig does not run independently; instead, it is invoked and managed by the Java program.

2. Pig API (PigServer): PigServer acts as a bridge between Java and Pig. It is a Pig API class that allows the Java program to load data, execute Pig Latin commands and register .pig scripts. No Pig shell is involved in this mode. PigServer translates Java instructions into Pig operations.

3. Pig Latin Script: The PigServer receives Pig Latin commands or scripts from the Java program and prepares them for execution. These scripts define the data processing logic.

4. Execution (MapReduce or Local): Pig automatically converts the Pig Latin script into MapReduce jobs for execution on Hadoop or runs them in local mode, depending on the configuration.

User Defined Function:

1. Apache Pig supports User Defined Functions (UDFs) along with built-in functions, allowing users to write custom processing logic in Pig Latin scripts.

2. Pig supports UDFs in multiple languages such as Java, Jython, Python, JavaScript, Ruby and Groovy, but Java provides full and efficient support since Pig is written in Java.

3. Using Java, UDFs can be created for all stages of data processing, including data loading, storing, transformation and aggregation, with better performance than other languages.

4. Apache Pig provides a Java UDF repository called Piggybank, which allows users to reuse existing UDFs and contribute their own.

5. Types of Pig UDFs in Java:

i. Filter Functions: Used in FILTER statements as conditions. They take a Pig value as input and return a Boolean value (true or false).

ii. Eval Functions: Used in FOREACH ... GENERATE statements. They take a Pig value as input and return a processed Pig result.

iii. Algebraic Functions: Used in FOREACH ... GENERATE statements on inner bags. They perform full MapReduce operations such as sum, count and average.